Jacobian Descent For Multi-Objective Optimization

Valérian Rey — Simplex Lab
valerian.rey@gmail.com
Joint work with Pierre Quinton

February 10, 2026

GitHub
github.com/TorchJD/torchjd

TorchJD
torchjd.org

arXiv
arxiv.org/pdf/2406.16232

Table of contents

Theory

Multi-objective optimization

Define the partial order $(\mathbb{R}^m, \preceq)$, for $\boldsymbol{u}, \boldsymbol{v} \in \mathbb{R}^m$, as

$$\begin{aligned} & \boldsymbol{u} \preceq \boldsymbol{v} & \\ \Leftrightarrow \quad & u_i \leq v_i & \forall i \in [m] \end{aligned}$$

Multi-objective optimization

Define the partial order $(\mathbb{R}^m, \preceq)$, for $\boldsymbol{u}, \boldsymbol{v} \in \mathbb{R}^m$, as

$$\begin{aligned} & \boldsymbol{u} \preceq \boldsymbol{v} & \\ \Leftrightarrow \quad & u_i \leq v_i & \forall i \in [m] \end{aligned}$$

Let $\boldsymbol{f} : \mathbb{R}^n \to \mathbb{R}^m$, the goal of multi-objective optimization is to find a **Pareto optimal** point $\boldsymbol{x}^*$ of $\boldsymbol{f}$.

Multi-objective optimization

Define the partial order $(\mathbb{R}^m, \preceq)$, for $\boldsymbol{u}, \boldsymbol{v} \in \mathbb{R}^m$, as

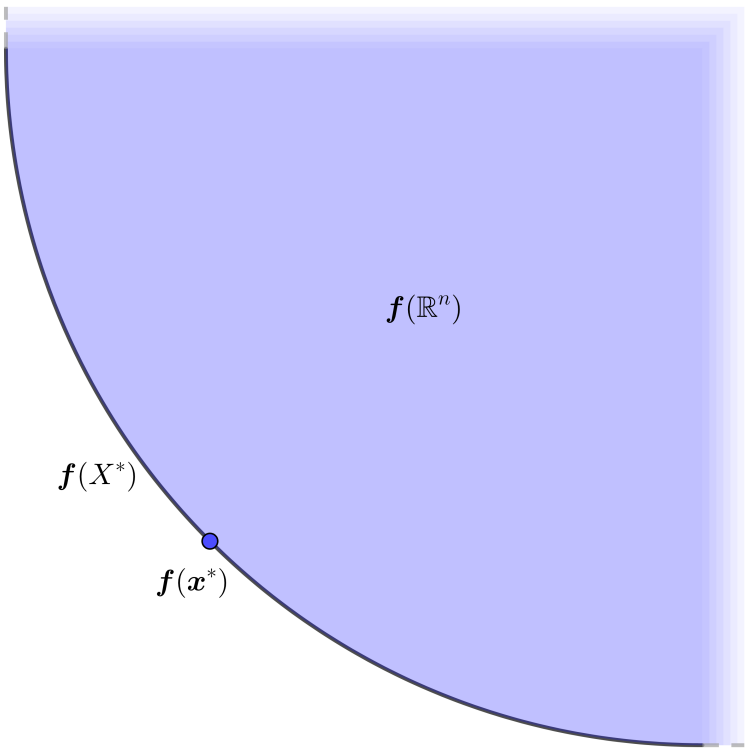
$$\begin{aligned} & \boldsymbol{u} \preceq \boldsymbol{v} & \\ \Leftrightarrow \quad & u_i \leq v_i & \forall i \in [m] \end{aligned}$$

Let $\boldsymbol{f} : \mathbb{R}^n \to \mathbb{R}^m$, the goal of multi-objective optimization is to find a **Pareto optimal** point $\boldsymbol{x}^*$ of $\boldsymbol{f}$.

$$\boldsymbol{x}^* \in X^* \quad \Leftrightarrow \quad \forall \boldsymbol{y} \in \mathbb{R}^n,\ \boldsymbol{f}(\boldsymbol{y}) \preceq \boldsymbol{f}(\boldsymbol{x}^*) \to \boldsymbol{f}(\boldsymbol{y}) = \boldsymbol{f}(\boldsymbol{x}^*)$$

Pareto optimality

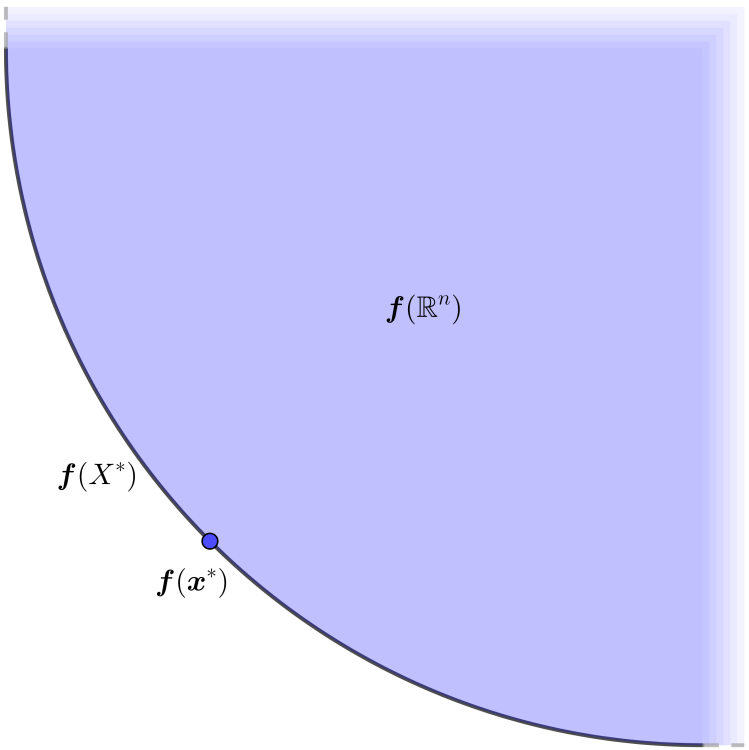
$$\boldsymbol{x}^* \in X^* \quad \Leftrightarrow \quad \forall \boldsymbol{y} \in \mathbb{R}^n,\ \boldsymbol{f}(\boldsymbol{y}) \preceq \boldsymbol{f}(\boldsymbol{x}^*) \rightarrow \boldsymbol{f}(\boldsymbol{y}) = \boldsymbol{f}(\boldsymbol{x}^*)$$



Pareto optimality

$$\boldsymbol{x}^* \in X^* \quad \Leftrightarrow \quad \forall \boldsymbol{y} \in \mathbb{R}^n,\ \boldsymbol{f}(\boldsymbol{y}) \preceq \boldsymbol{f}(\boldsymbol{x}^*) \rightarrow \boldsymbol{f}(\boldsymbol{y}) = \boldsymbol{f}(\boldsymbol{x}^*)$$

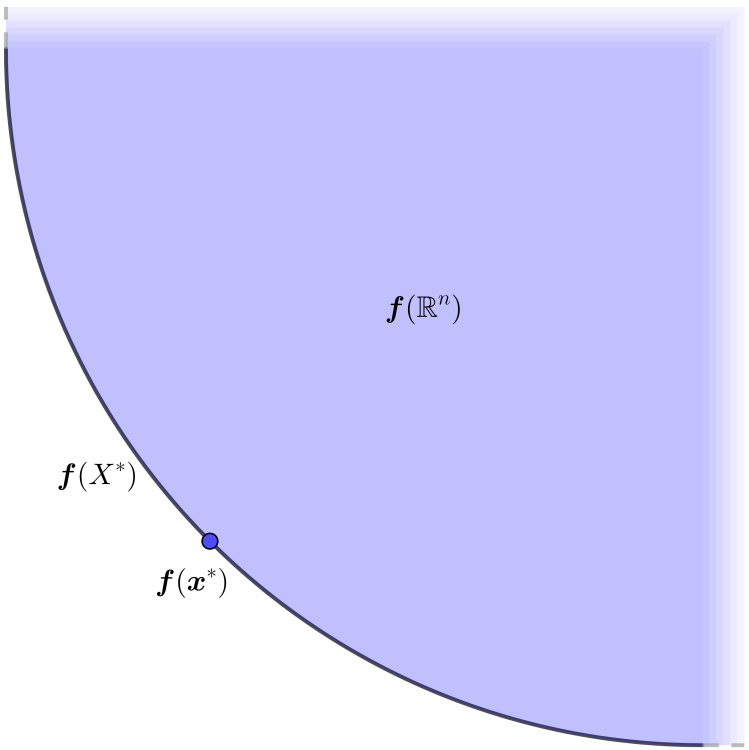
Pareto set: X^*



Pareto optimality

$$\boldsymbol{x}^* \in X^* \quad \Leftrightarrow \quad \forall \boldsymbol{y} \in \mathbb{R}^n,\ \boldsymbol{f}(\boldsymbol{y}) \preceq \boldsymbol{f}(\boldsymbol{x}^*) \rightarrow \boldsymbol{f}(\boldsymbol{y}) = \boldsymbol{f}(\boldsymbol{x}^*)$$

Pareto set: X^*
Pareto Front: $\boldsymbol{f}(X^*)$



First-order Taylor approximation

For any $\boldsymbol{x} \in \mathbb{R}^n$, the Jacobian of $\boldsymbol{f}$ at $\boldsymbol{x}$, is

$$\mathcal{J}\boldsymbol{f}(\boldsymbol{x}) = \begin{bmatrix} \nabla f_1(\boldsymbol{x})^\top \\ \nabla f_2(\boldsymbol{x})^\top \\ \vdots \\ \nabla f_m(\boldsymbol{x})^\top \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_1(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\boldsymbol{x}) \\ \frac{\partial}{\partial x_1} f_2(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_2(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_2(\boldsymbol{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_m(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\boldsymbol{x}) \end{bmatrix}$$

First-order Taylor approximation

For any $\boldsymbol{x} \in \mathbb{R}^n$, the Jacobian of $\boldsymbol{f}$ at $\boldsymbol{x}$, is

$$\mathcal{J}\boldsymbol{f}(\boldsymbol{x}) = \begin{bmatrix} \nabla f_1(\boldsymbol{x})^\top \\ \nabla f_2(\boldsymbol{x})^\top \\ \vdots \\ \nabla f_m(\boldsymbol{x})^\top \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_1(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\boldsymbol{x}) \\ \frac{\partial}{\partial x_1} f_2(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_2(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_2(\boldsymbol{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_m(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\boldsymbol{x}) \end{bmatrix}$$

For a small enough update $\boldsymbol{y} \in \mathbb{R}^n$, Taylor's theorem states

$$\boldsymbol{f}(\boldsymbol{x} + \boldsymbol{y}) \approx \boldsymbol{f}(\boldsymbol{x}) + \mathcal{J}\boldsymbol{f}(\boldsymbol{x}) \cdot \boldsymbol{y}$$

First-order Taylor approximation

For any $\boldsymbol{x} \in \mathbb{R}^n$, the Jacobian of $\boldsymbol{f}$ at $\boldsymbol{x}$, is

$$\mathcal{J}\boldsymbol{f}(\boldsymbol{x}) = \begin{bmatrix} \nabla f_1(\boldsymbol{x})^\top \\ \nabla f_2(\boldsymbol{x})^\top \\ \vdots \\ \nabla f_m(\boldsymbol{x})^\top \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_1(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\boldsymbol{x}) \\ \frac{\partial}{\partial x_1} f_2(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_2(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_2(\boldsymbol{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_m(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\boldsymbol{x}) \end{bmatrix}$$

For a small enough update $\boldsymbol{y} \in \mathbb{R}^n$, Taylor's theorem states

$$\boldsymbol{f}(\boldsymbol{x} + \boldsymbol{y}) \approx \boldsymbol{f}(\boldsymbol{x}) + \mathcal{J}\boldsymbol{f}(\boldsymbol{x}) \cdot \boldsymbol{y}$$

We select $\boldsymbol{y} = -\eta \mathcal{A}(\mathcal{J}\boldsymbol{f}(\boldsymbol{x})) \in \mathbb{R}^n$.

Jacobian descent

The mapping $\mathcal{A} : \mathbb{R}^{m \times n} \to \mathbb{R}^n$ is called an **aggregator**, it parameterizes **Jacobian descent**:

Jacobian descent

The mapping $\mathcal{A} : \mathbb{R}^{m \times n} \to \mathbb{R}^n$ is called an **aggregator**, it parameterizes **Jacobian descent**:

Algorithm 1: Jacobian descent with aggregator $\mathcal{A}$

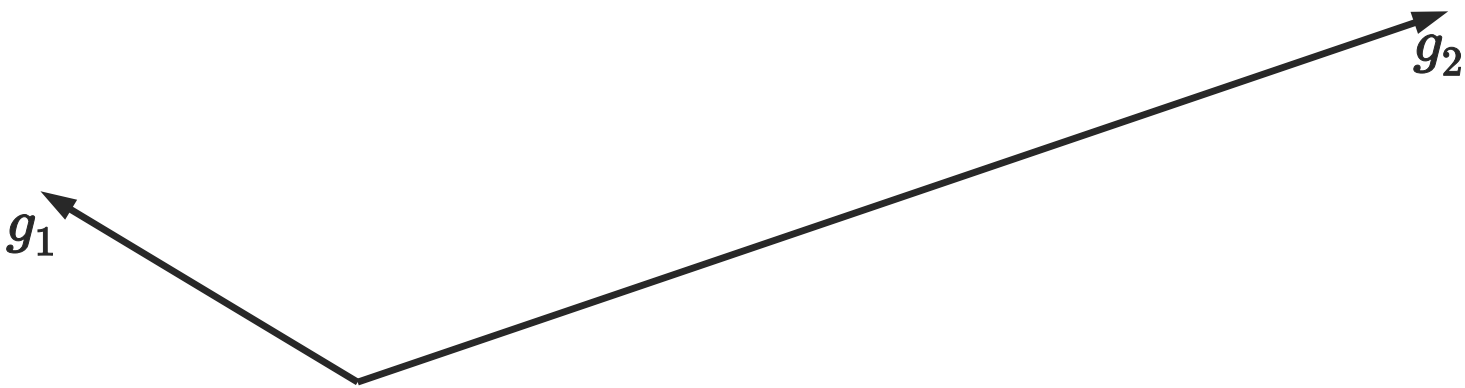
Input: $\boldsymbol{x} \in \mathbb{R}^n$, $0 < \eta$, $T \in \mathbb{N}$, $\mathcal{A} : \mathbb{R}^{m \times n} \to \mathbb{R}^n$

for $t \leftarrow 1$ **to** T **do**

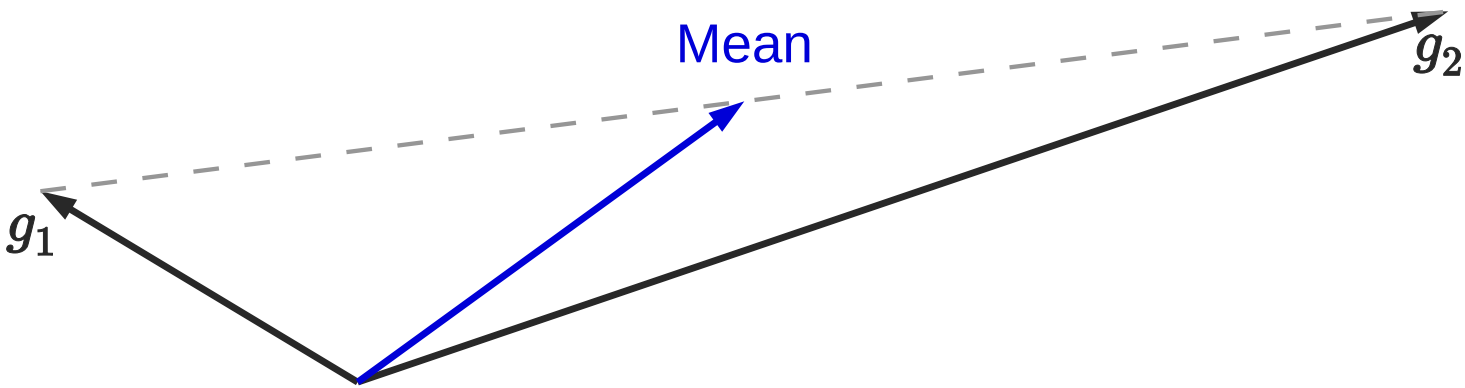
$\quad \boldsymbol{x} \leftarrow \boldsymbol{x} - \eta \mathcal{A}(\mathcal{J}\boldsymbol{f}(\boldsymbol{x}))$

Output: $\boldsymbol{x}$

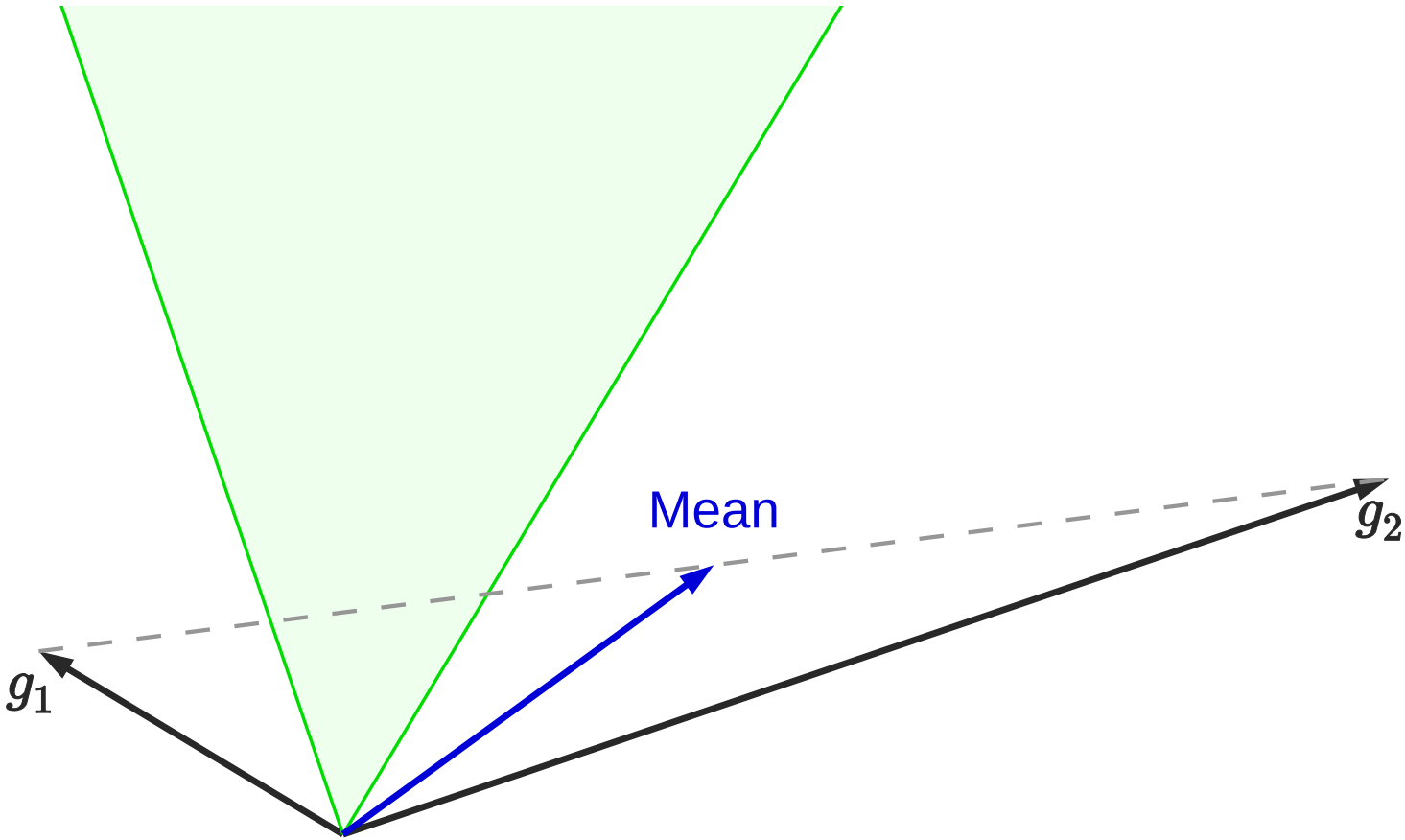
Unconflicting projection of gradients (UPGrad)



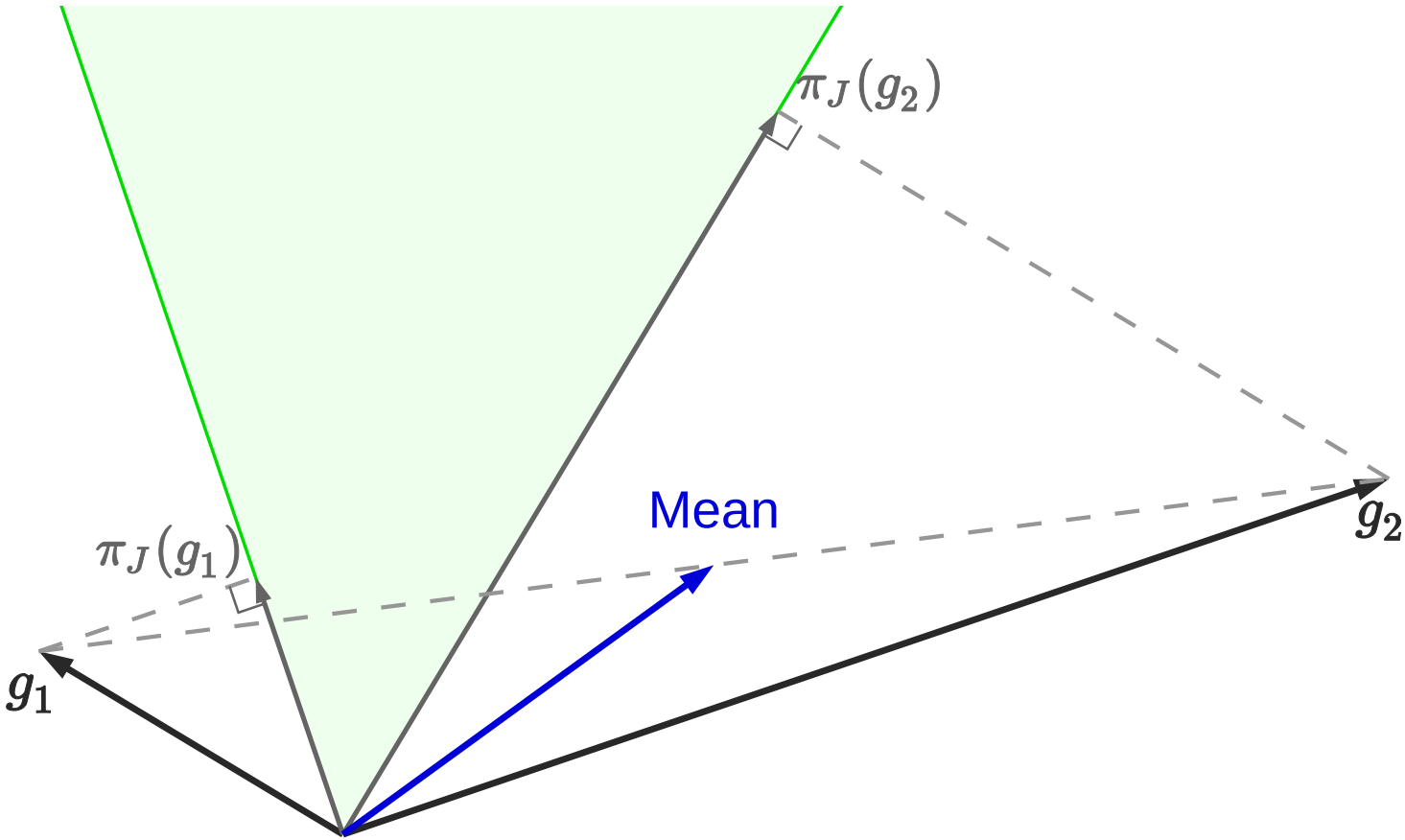
Unconflicting projection of gradients (UPGrad)



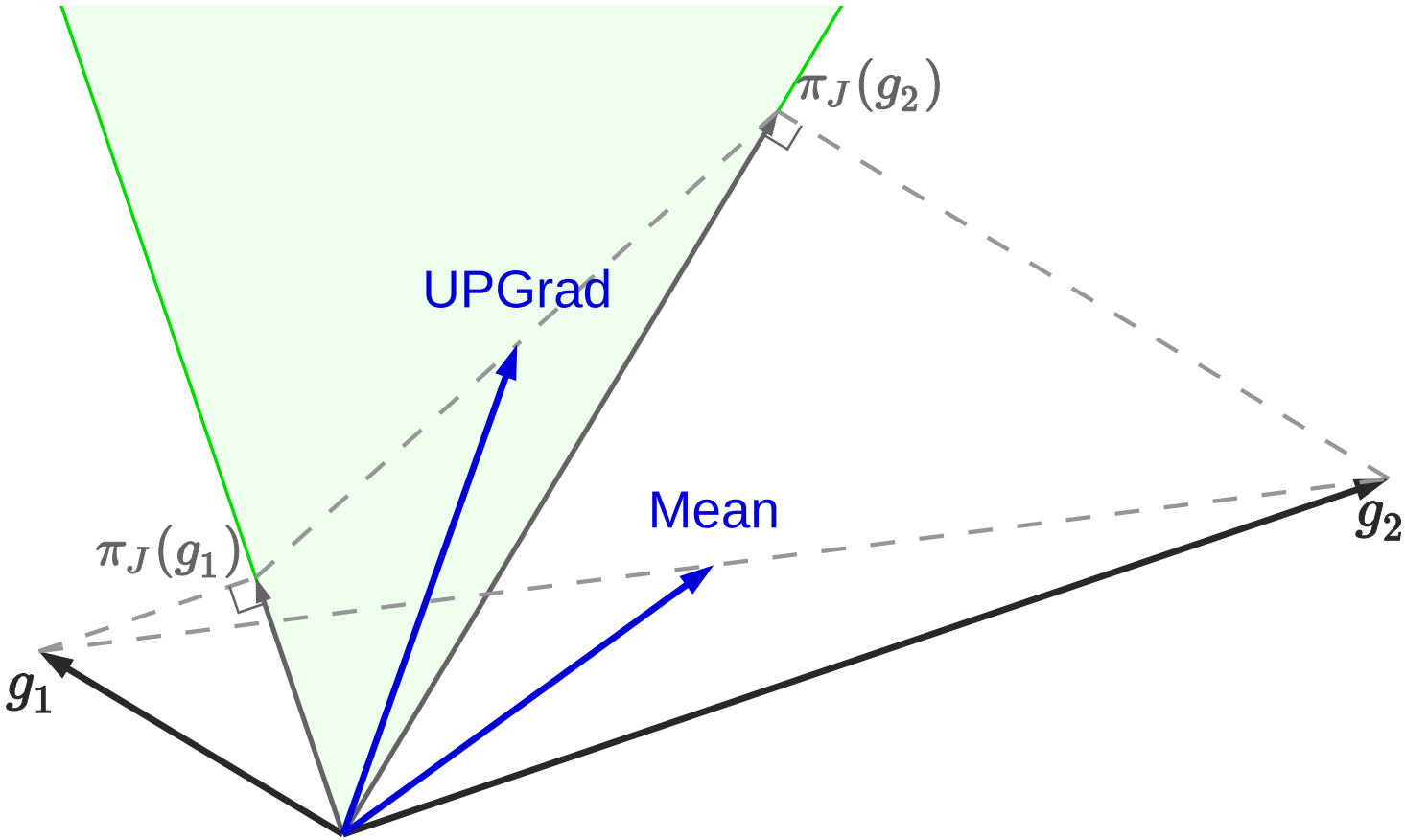
Unconflicting projection of gradients (UPGrad)



Unconflicting projection of gradients (UPGrad)



Unconflicting projection of gradients (UPGrad)



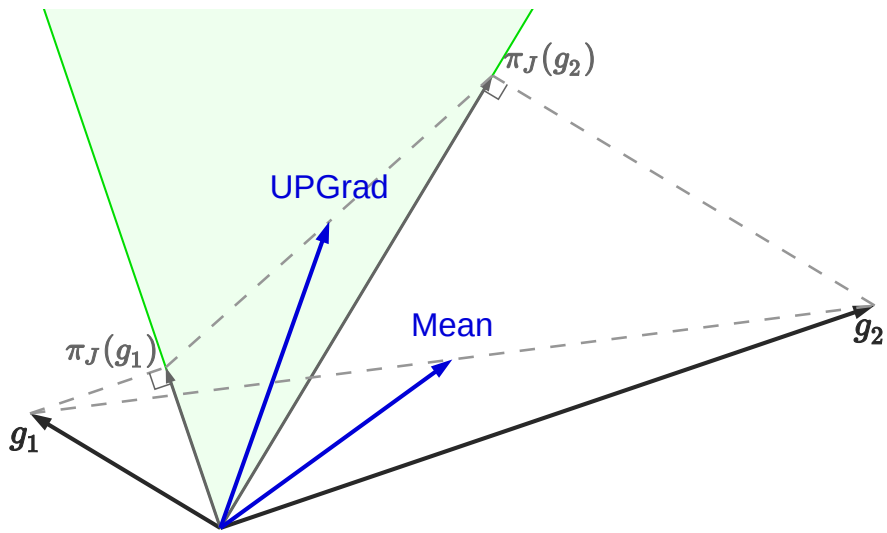
Formal definition of UPGrad

Given $J \in \mathbb{R}^{m \times n}$, let C_J be the cone $\{\boldsymbol{y} \in \mathbb{R}^n : \boldsymbol{0} \preceq J\boldsymbol{y}\}$. $\mathcal{A}$ is non-conflicting if for all $J \in \mathbb{R}^{m \times n}$, $\mathcal{A}(J) \in C_J$.

For any $\boldsymbol{x} \in \mathbb{R}^n$, the projection of $\boldsymbol{x}$ onto C_J is

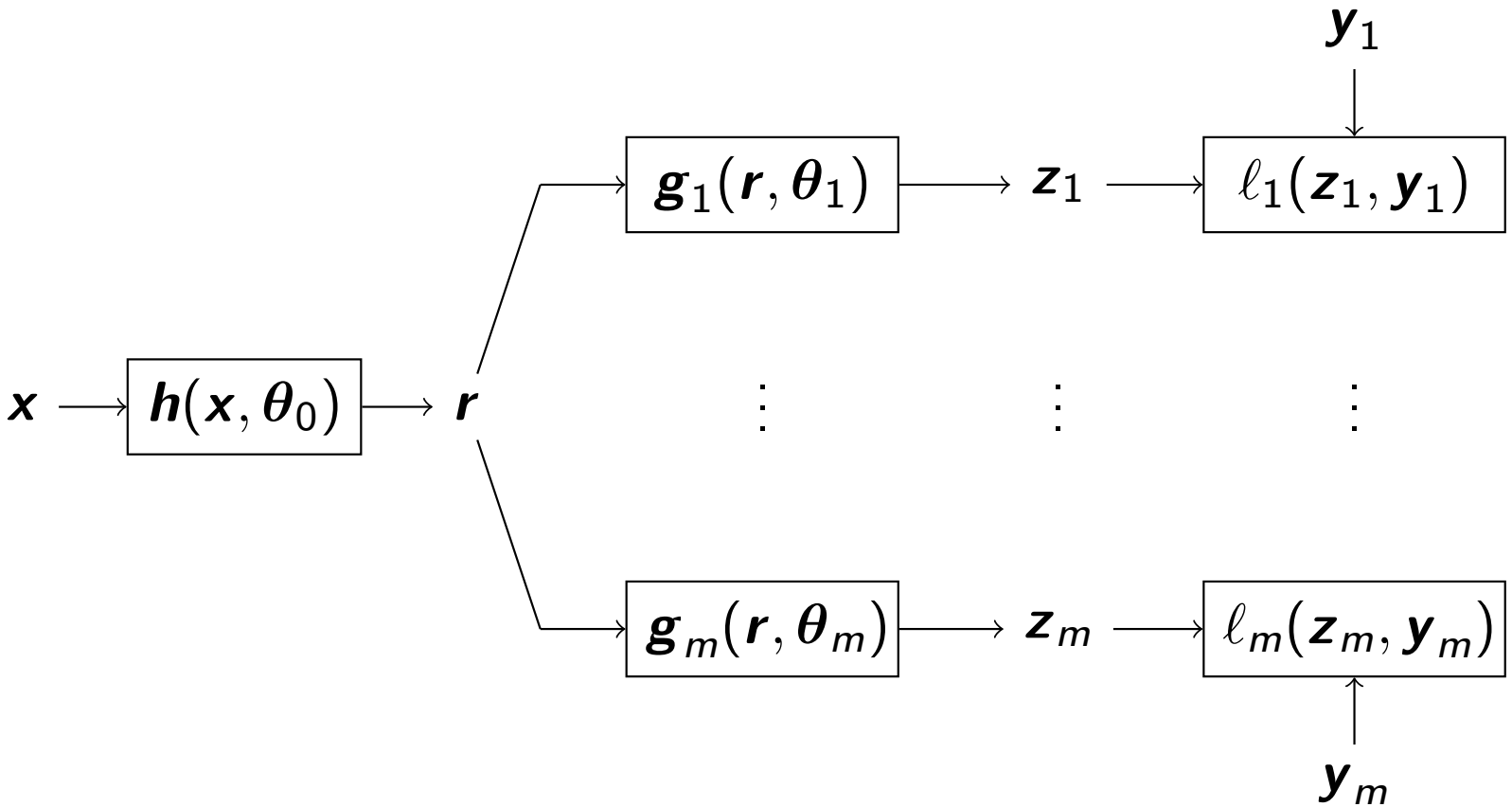
$$\pi_J(\boldsymbol{x}) = \underset{\boldsymbol{y} \in C_J}{\arg\min} \|\boldsymbol{x} - \boldsymbol{y}\|^2$$

$$\mathcal{A}_{\text{UPGrad}}(J) = \frac{1}{m} \sum_{i \in [m]} \pi_J\left(J^\top \boldsymbol{e}_i\right)$$

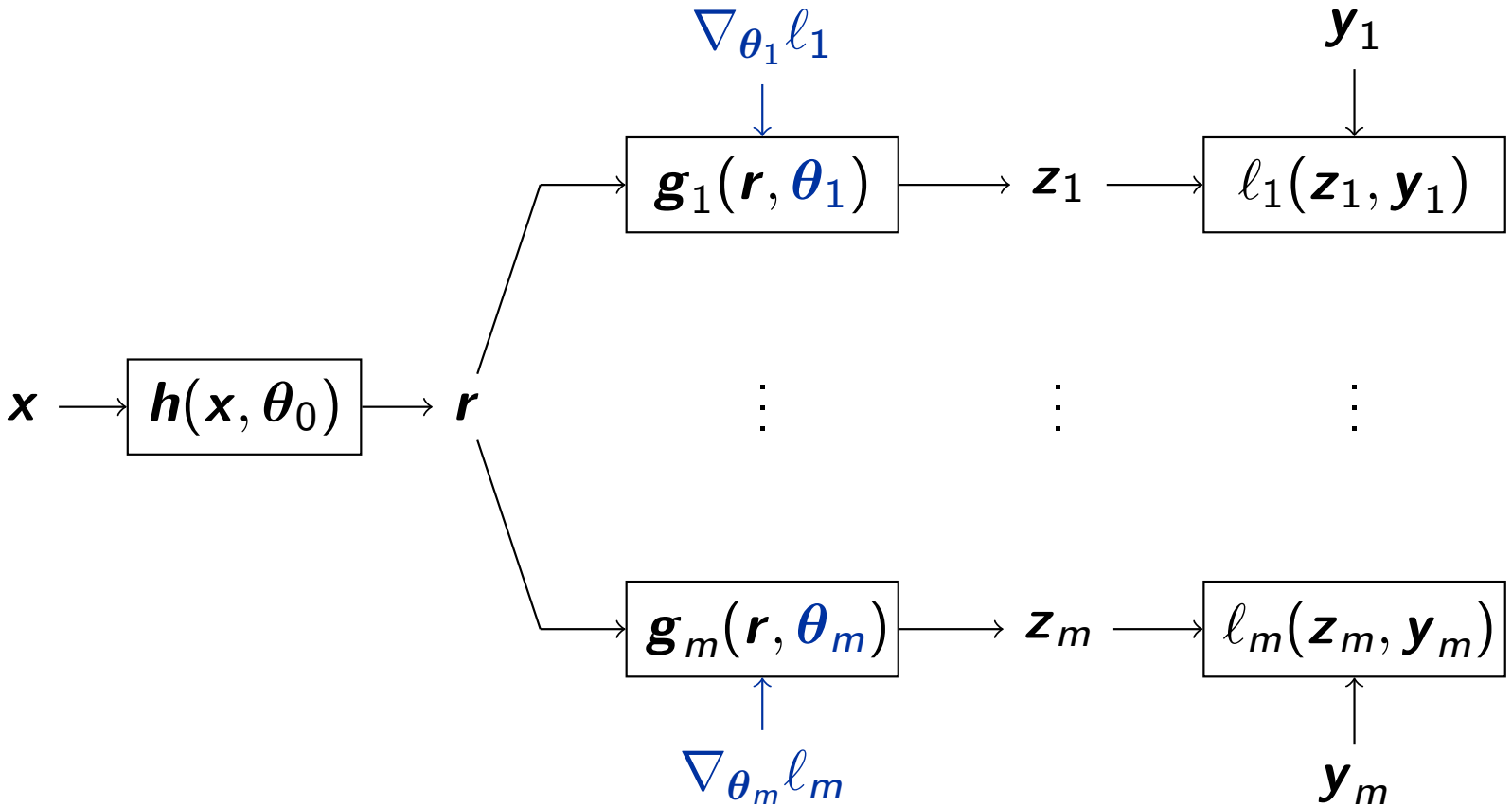


Applications and experimentation

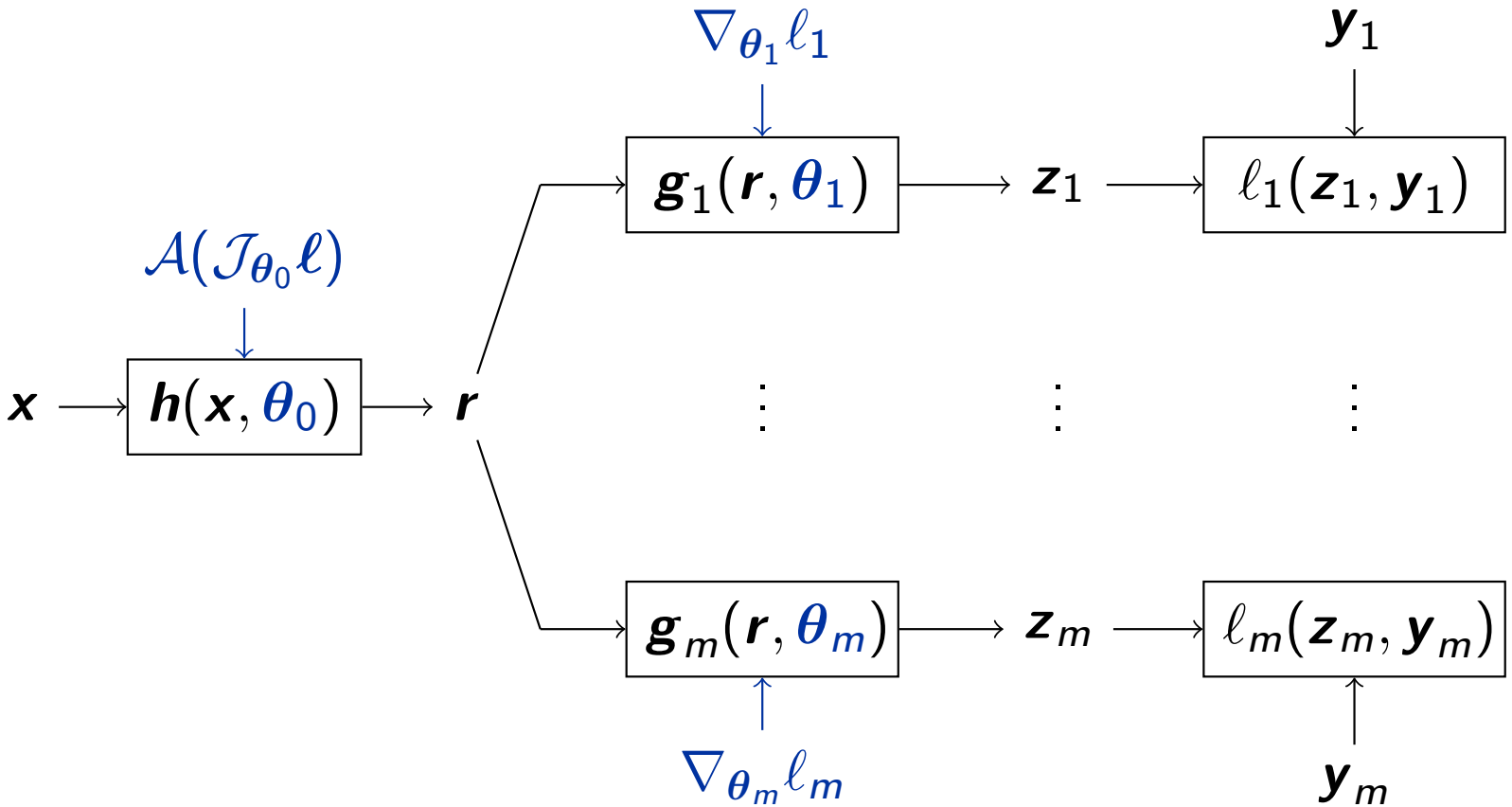
Multi-task learning



Multi-task learning



Multi-task learning



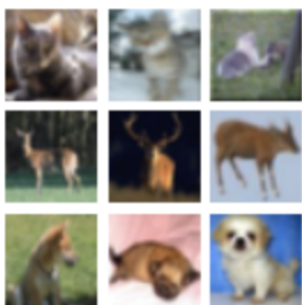
Experimental results: per-sample JD

Each element of the batch has its own loss.

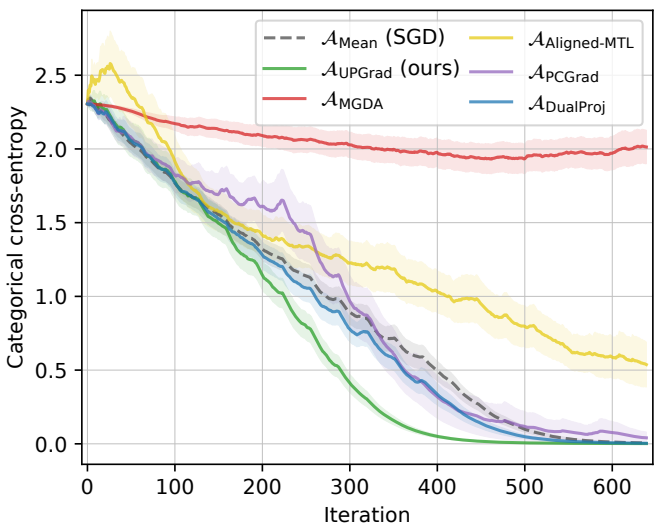
Experimental results: per-sample JD

Each element of the batch has its own loss.

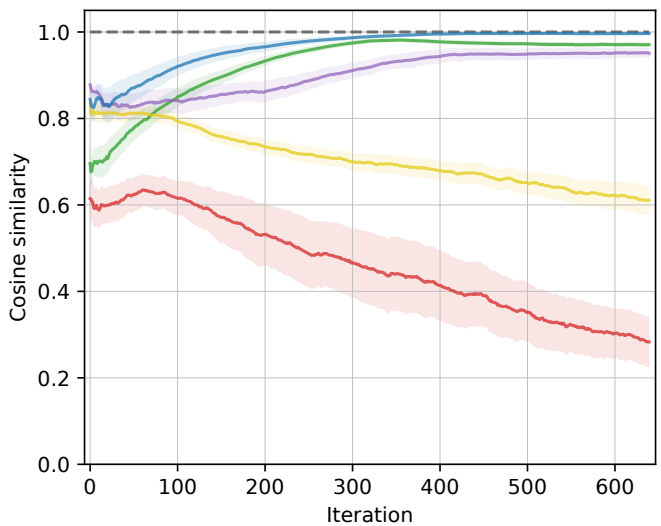
CIFAR-10



Training Loss



Update Similarity



Future directions

Future directions

Implementation

- Make the `autogram` engine faster.

Future directions

Implementation

- Make the `autogram` engine faster.
- Make a better implementation of $\mathcal{A}_{\text{UPGrad}}$.

Future directions

Implementation

- Make the `autogram` engine faster.
- Make a better implementation of $\mathcal{A}_{\text{UPGrad}}$.
- Improve the interface of TorchJD.

Future directions

Implementation

- Make the `autogram` engine faster.
- Make a better implementation of $\mathcal{A}_{\text{UPGrad}}$.
- Improve the interface of TorchJD.

Applications

- Are there multi-task learning problems with actual conflict?

Future directions

Implementation

- Make the `autogram` engine faster.
- Make a better implementation of $\mathcal{A}_{\text{UPGrad}}$.
- Improve the interface of TorchJD.

Applications

- Are there multi-task learning problems with actual conflict?
- Could JD (per-sample) help training diffusion models?

Future directions

Implementation

- Make the `autogram` engine faster.
- Make a better implementation of $\mathcal{A}_{\text{UPGrad}}$.
- Improve the interface of TorchJD.

Applications

- Are there multi-task learning problems with actual conflict?
- Could JD (per-sample) help training diffusion models?
- Could JD (per-depth) help training recurrent models?

Future directions

Implementation

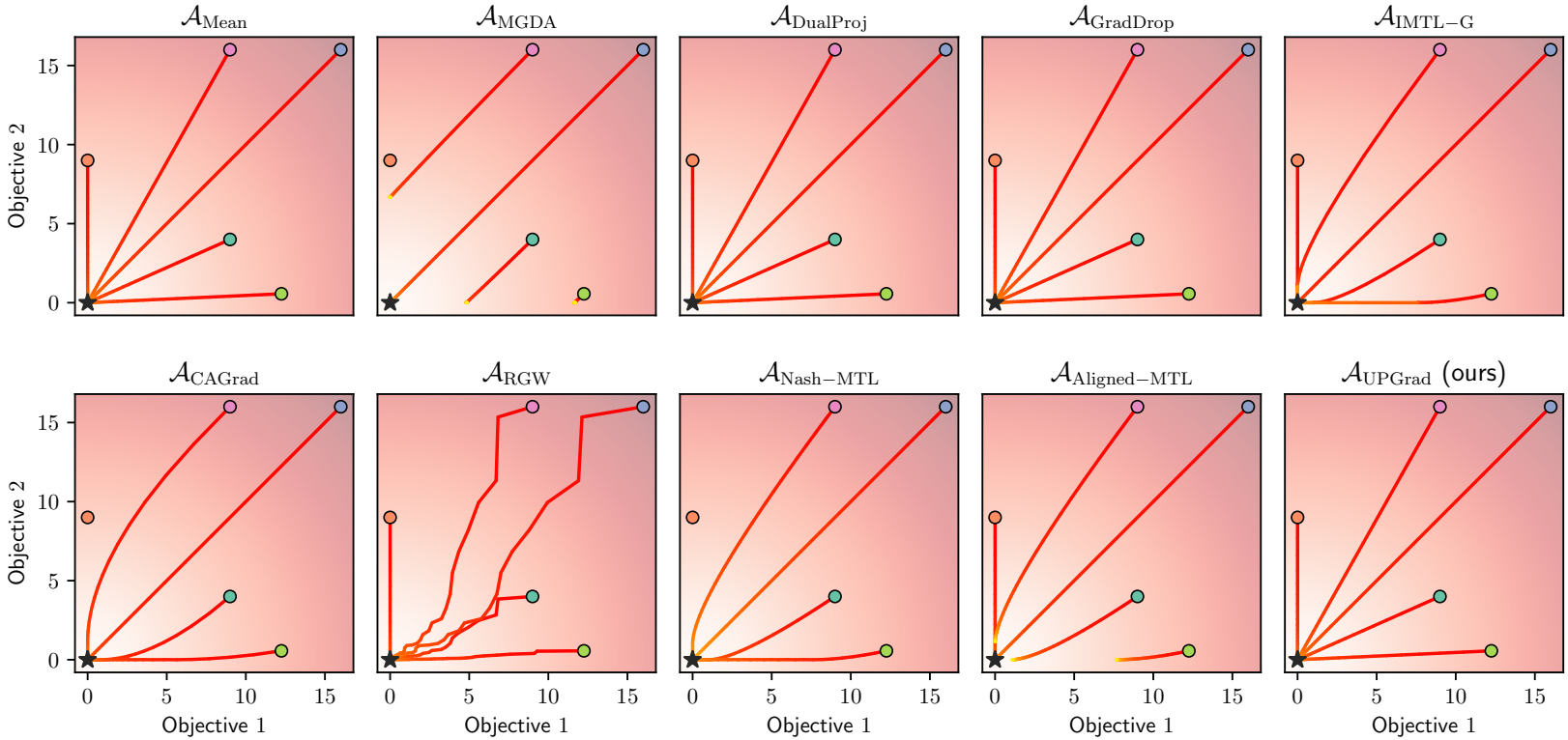
- Make the `autogram` engine faster.
- Make a better implementation of $\mathcal{A}_{\text{UPGrad}}$.
- Improve the interface of TorchJD.

Applications

- Are there multi-task learning problems with actual conflict?
- Could JD (per-sample) help training diffusion models?
- Could JD (per-depth) help training recurrent models?
- Find new applications.

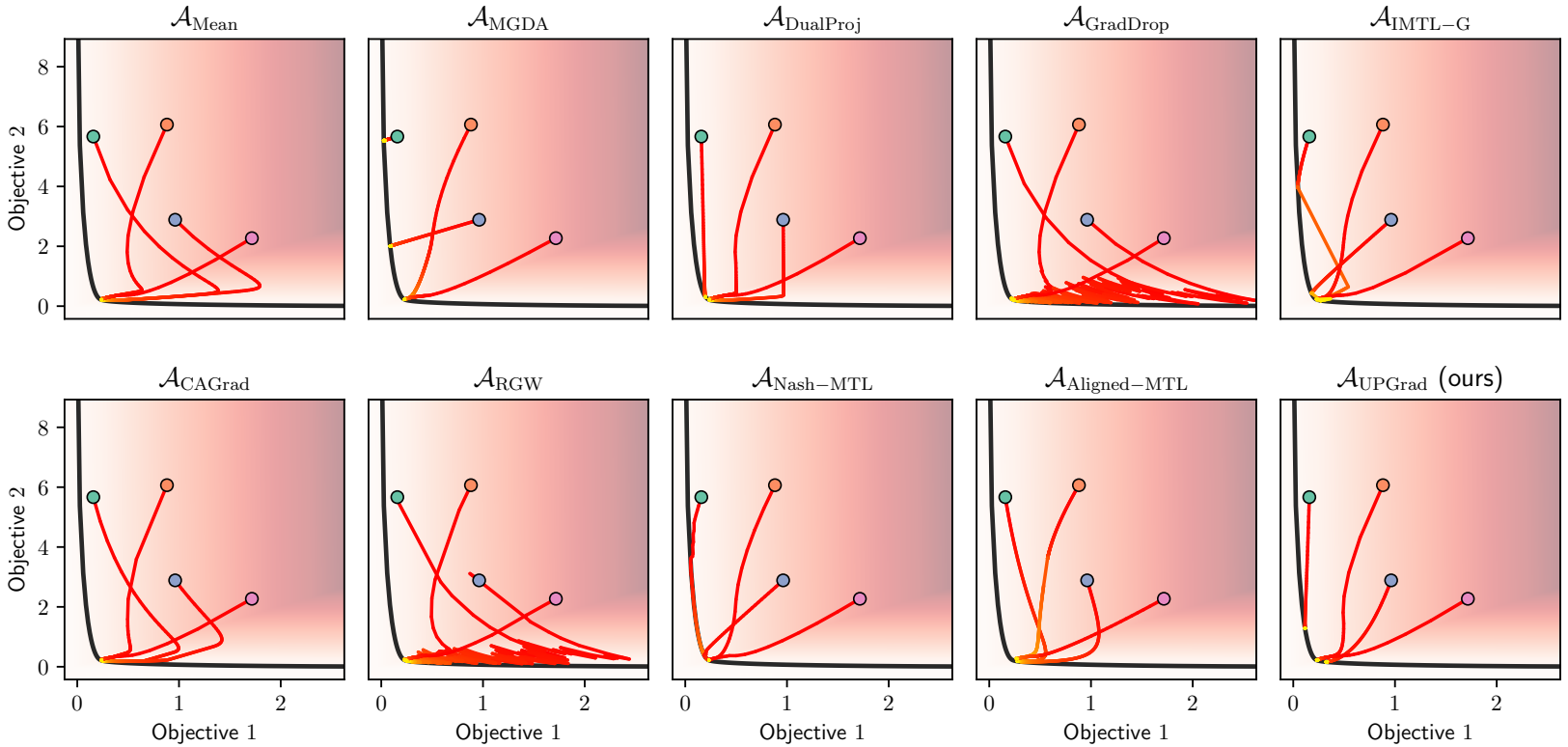
Bonus: Element-wise quadratic function trajectories

Minimize $\boldsymbol{f}$: $\begin{bmatrix} x \\ y \end{bmatrix} \mapsto \begin{bmatrix} x^2 \\ y^2 \end{bmatrix}$. It's Jacobian at $\begin{bmatrix} x \\ y \end{bmatrix}$ is $\begin{bmatrix} 2x & 0 \\ 0 & 2y \end{bmatrix}$.



Bonus: Convex quadratic form trajectories

Some convex quadratic function $\boldsymbol{f} : \mathbb{R}^2 \to \mathbb{R}^2$, with highly conflicting gradients.



Bonus: Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\boldsymbol{f}(\boldsymbol{x}))$ is a linear combination of the rows of $\mathcal{J}\boldsymbol{f}(\boldsymbol{x})$.

Bonus: Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\boldsymbol{f}(\boldsymbol{x}))$ is a linear combination of the rows of $\mathcal{J}\boldsymbol{f}(\boldsymbol{x})$.

The weights only depend on the (small) Gramian $\mathcal{G}\boldsymbol{f}(\boldsymbol{x}) = \mathcal{J}\boldsymbol{f}(\boldsymbol{x}) \cdot \mathcal{J}\boldsymbol{f}(\boldsymbol{x})^\top$.

Bonus: Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\boldsymbol{f}(\boldsymbol{x}))$ is a linear combination of the rows of $\mathcal{J}\boldsymbol{f}(\boldsymbol{x})$.

The weights only depend on the (small) Gramian $\mathcal{G}\boldsymbol{f}(\boldsymbol{x}) = \mathcal{J}\boldsymbol{f}(\boldsymbol{x}) \cdot \mathcal{J}\boldsymbol{f}(\boldsymbol{x})^\top$.

We can compute $\mathcal{G}\boldsymbol{f}(\boldsymbol{x})$ directly, extract the weights, combine the losses, and do a step of gradient descent.

Bonus: Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\boldsymbol{f}(\boldsymbol{x}))$ is a linear combination of the rows of $\mathcal{J}\boldsymbol{f}(\boldsymbol{x})$.

The weights only depend on the (small) Gramian $\mathcal{G}\boldsymbol{f}(\boldsymbol{x}) = \mathcal{J}\boldsymbol{f}(\boldsymbol{x}) \cdot \mathcal{J}\boldsymbol{f}(\boldsymbol{x})^\top$.

We can compute $\mathcal{G}\boldsymbol{f}(\boldsymbol{x})$ directly, extract the weights, combine the losses, and do a step of gradient descent.

We have developed the `autogram` engine to compute $\mathcal{G}\boldsymbol{f}(\boldsymbol{x})$ efficiently.

Bonus: Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\boldsymbol{f}(\boldsymbol{x}))$ is a linear combination of the rows of $\mathcal{J}\boldsymbol{f}(\boldsymbol{x})$.

The weights only depend on the (small) Gramian $\mathcal{G}\boldsymbol{f}(\boldsymbol{x}) = \mathcal{J}\boldsymbol{f}(\boldsymbol{x}) \cdot \mathcal{J}\boldsymbol{f}(\boldsymbol{x})^\top$.

We can compute $\mathcal{G}\boldsymbol{f}(\boldsymbol{x})$ directly, extract the weights, combine the losses, and do a step of gradient descent.

We have developed the `autogram` engine to compute $\mathcal{G}\boldsymbol{f}(\boldsymbol{x})$ efficiently.

This saves a lot of memory, so it's often much faster.

Bonus: Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\boldsymbol{f}(\boldsymbol{x}))$ is a linear combination of the rows of $\mathcal{J}\boldsymbol{f}(\boldsymbol{x})$.

The weights only depend on the (small) Gramian $\mathcal{G}\boldsymbol{f}(\boldsymbol{x}) = \mathcal{J}\boldsymbol{f}(\boldsymbol{x}) \cdot \mathcal{J}\boldsymbol{f}(\boldsymbol{x})^\top$.

We can compute $\mathcal{G}\boldsymbol{f}(\boldsymbol{x})$ directly, extract the weights, combine the losses, and do a step of gradient descent.

We have developed the `autogram` engine to compute $\mathcal{G}\boldsymbol{f}(\boldsymbol{x})$ efficiently.

This saves a lot of memory, so it's often much faster.

Applicable to most other aggregators from the literature.